

```

onPageLoad()
var tasks = []
var current_task_id = 1
const form = document.getElementById("form")

form.addEventListener("submit",function (e){
    e.preventDefault()

    const name =
document.getElementById("NameofTask").value
    const description =

Show more ▾

```

Your code has a few bugs that will likely cause errors or unexpected behavior.

1. onPageLoad() can crash if no tasks exist

```

tasks = JSON.parse(localStorage.getItem("tasks"))
for(const task of tasks){
    // ...
}

```

If "tasks" is not in localStorage, JSON.parse() returns null, and for...of null throws an error.

Fix:

```

function onPageLoad() {
    tasks = JSON.parse(localStorage.getItem("tasks")) || [];

    for (const task of tasks) {
        addTask(
            task.id,
            task.name,
            task.description,
            task.startDate,
            task.endDate
        );
    }
}

```

```
    );  
  }  
}
```

2. Wrong arguments passed to `addTask()` in `onPageLoad()`

Your function definition is:

```
JavaScript
```

But you call it like this:

```
JavaScript
```

Problems:

- Missing `id`
- Property names don't match (`startDate` doesn't exist)
- You stored `startDate` and `endDate`

Fix:

```
addTask(  
  task.id,  
  task.name,  
  task.description,  
  task.startDate,  
  task.endDate  
);
```

3. `current_task_id` is not restored after page reload

After refreshing the page:

JavaScript

so new tasks may reuse existing IDs.

Fix:

Update it when loading tasks:

```
function onPageLoad() {
  tasks = JSON.parse(localStorage.getItem("tasks")) || [];

  if (tasks.length > 0) {
    current_task_id = Math.max(...tasks.map(task => task.id));
  }

  for (const task of tasks) {
    addTask(
      task.id,
      task.name,
      task.description,
      task.startDate,
      task.endDate
    );
  }
}
```

4. newId should be declared

Current code:

```
function getNewId(){
  newId = current task id + 1
  return newId
}
```

This creates a global variable accidentally.

Fix:

```
function getNewId() {  
    const newId = current_task_id + 1;  
    current_task_id = newId;  
    return newId;  
}  
  
JavaScript
```

5. First task gets ID 2

Because:

JavaScript

and

JavaScript

The first generated ID becomes 2 .

If you want the first task to be 1 :

JavaScript

or

```
function getNewId() {  
    return ++current_task_id;  
}
```

Corrected onPageLoad()

```
function onPageLoad() {
  tasks = JSON.parse(localStorage.getItem("tasks")) || [];

  if (tasks.length > 0) {
    current_task_id = Math.max(...tasks.map(task => task.id));
  }

  for (const task of tasks) {
    addTask(
      task.id,
      task.name,
      task.description,
      task.startDate,
      task.endDate
    );
  }
}
```

If you're still seeing a specific error in the browser console, paste the exact error message and I can pinpoint the line causing it.

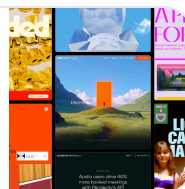


Framer
Sponsored



Build Better Websites, Faster

Launch websites with enterprise needs at startup speeds.



i want to delete the task when the delete button is clicked
and to remove the tr of that task

You can attach a click event listener to the **Delete** button when you create the row. When clicked:

1. Remove the task from the `tasks` array.
2. Update `localStorage`.
3. Remove the `<tr>` from the DOM.

Update your addTask() function

```
function addTask(id, name, description, startDate, endDate) {
  const tr = document.createElement("tr");
  tr.id = id;

  tr.innerHTML = `
    <td>${name}</td>
    <td>${description}</td>
    <td>${startDate}</td>
    <td>${endDate}</td>
    <td><button class="update-button">Update</button></td>
    <td><button class="delete-button">Delete</button></td>
  `;

  const deleteButton = tr.querySelector(".delete-button");

  deleteButton.addEventListener("click", function () {
    // Remove task from array
    tasks = tasks.filter(task => task.id !== id);

    // Update localStorage
    localStorage.setItem("tasks", JSON.stringify(tasks));

    // Remove row from table
    tr.remove();
  });

  const table = document.getElementById("table");
  table.appendChild(tr);
}
```

Alternative using the row ID

If you prefer finding the row by its ID:

```
deleteButton.addEventListener("click", function () {
  tasks = tasks.filter(task => task.id !== id);

  localStorage.setItem("tasks", JSON.stringify(tasks));
});
```

```

        document.getElementById(id).remove();
    }
}

JavaScript

```

The first approach (`tr.remove()`) is simpler because you already have a reference to the row that was clicked.

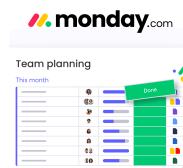


Monday.com
Sponsored



When team tasks get messy

Keep team tasks clear, owned, and moving on monday.com.



and the update button

For the **Update** button, a common approach is:

1. When the user clicks **Update**, fill the form with that task's current data.
2. Remember which task is being edited.
3. When the form is submitted, update the existing task instead of creating a new one.
4. Update the row in the table and save to `localStorage`.

Add a variable to track the task being edited

JavaScript

Update button listener inside `addTask()`

```

const updateButton = tr.querySelector(".update-button");

updateButton.addEventListener("click", function () {
    document.getElementById("NameofTask").value = name;
    document.getElementById("DescriptionofTask").value = description;

```

```
document.getElementById("StartofTask").value = stDate;
document.getElementById("EndofTask").value = endDate;

editingTaskId = id;
});
```

Modify your submit handler

Instead of always creating a new task:

```
form.addEventListener("submit", function (e) {
    e.preventDefault();

    const name = document.getElementById("NameofTask").value;
    const description = document.getElementById("DescriptionofTask").value;
    const stDate = document.getElementById("StartofTask").value;
    const endDate = document.getElementById("EndofTask").value;

    if (editingTaskId !== null) {
        // Update existing task
        const task = tasks.find(t => t.id === editingTaskId);

        task.name = name;
        task.description = description;
        task.startDate = stDate;
        task.endDate = endDate;

        localStorage.setItem("tasks", JSON.stringify(tasks));

        // Update row in table
        const tr = document.getElementById(editingTaskId);

        tr.children[0].textContent = name;
        tr.children[1].textContent = description;
        tr.children[2].textContent = stDate;
        tr.children[3].textContent = endDate;

        editingTaskId = null;
    } else {
        // Create new task
        const id = getNewId();

        tasks.push({
            id,
            name,
            description,
```

```

        startDate: stDate,
        endDate: endDate
    });

    localStorage.setItem("tasks", JSON.stringify(tasks));

    addTask(id, name, description, stDate, endDate);
}

form.reset();
});

```

One thing to watch out for: in your current code, the Update button listener captures the original values (name , description , etc.) from when the row was created. After an update, clicking Update again may show old values. A better design is to store only the `id` in the button handler and look up the current task from the `tasks` array each time. That avoids stale data.



```

<!DOCTYPE html>
<html>
  <link rel="stylesheet" href="styles.css">
<head>
</head>
<body>
  <div class="form-parent">
    <form id="form">
      <div class="input-fields">
        <div class="inputs">
          <label for="NameofTask">Name</label>

```

Show more ▾

A clean approach is to create a hidden modal in your HTML and show it when the **Update** button is clicked.

1. Add this modal below your table

```

<div id="updateModal" class="modal">
  <div class="modal-content">
    <h2>Update Task</h2>

    <input type="text" id="updateName" placeholder="Task Name">
    <input type="text" id="updateDescription" placeholder="Description">
    <input type="date" id="updateStart">
    <input type="date" id="updateEnd">

    <div class="modal-buttons">
      <button id="saveUpdate">Save</button>
      <button id="closeModal">Cancel</button>
    </div>
  </div>
</div>

```

2. Add CSS

```

.modal {
  display: none;
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.5);

  justify-content: center;
  align-items: center;
}

.modal-content {
  background: white;
  padding: 20px;
  border-radius: 8px;

  display: flex;
  flex-direction: column;
  gap: 10px;

  min-width: 300px;
}

.modal-buttons {

```

```
display: flex;  
gap: 10px;  
}
```

3. Add variables in JavaScript

```
let currentUpdateId = null;  
  
const modal = document.getElementById("updateModal");  
const saveButton = document.getElementById("saveUpdate");  
const closeButton = document.getElementById("closeModal");
```

4. Update your addTask() function

After creating the row:

```
const updateButton = tr.querySelector(".update-button");  
  
updateButton.addEventListener("click", function () {  
  
    const task = tasks.find(t => t.id === id);  
  
    currentUpdateId = id;  
  
    document.getElementById("updateName").value = task.name;  
    document.getElementById("updateDescription").value = task.description;  
    document.getElementById("updateStart").value = task.startDate;  
    document.getElementById("updateEnd").value = task.endDate;  
  
    modal.style.display = "flex";  
});
```

5. Close modal

```
closeButton.addEventListener("click", function () {  
    modal.style.display = "none";  
});
```

6. Save updates

```
saveButton.addEventListener("click", function () {  
  
    const task = tasks.find(t => t.id === currentUpdateId);  
  
    task.name = document.getElementById("updateName").value;  
    task.description = document.getElementById("updateDescription").value;  
    task.startDate = document.getElementById("updateStart").value;  
    task.endDate = document.getElementById("updateEnd").value;  
  
    localStorage.setItem("tasks", JSON.stringify(tasks));  
  
    const tr = document.getElementById(currentUpdateId);  
  
    tr.children[0].textContent = task.name;  
    tr.children[1].textContent = task.description;  
    tr.children[2].textContent = task.startDate;  
    tr.children[3].textContent = task.endDate;  
  
    modal.style.display = "none";  
});
```

Important

Make sure every row's ID is set correctly:

JavaScript

```
tr.id = id;
```

and when loading from storage:

JavaScript

```
addTask(  
    task.id,  
    task.name,  
    task.description,  
    task.startDate,  
    task.endDate  
);
```

This will give you a popup modal that opens with the current task values, allows editing them, updates the table row immediately, and persists the changes in `localStorage`.

